HW8 Optimal FIR design

1.

Our algorithm follows that steps

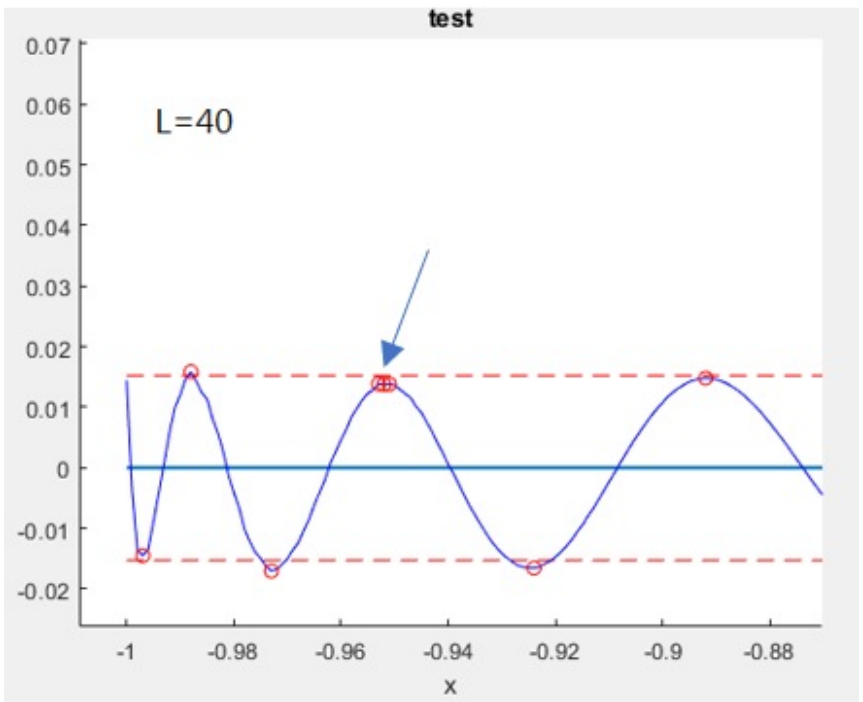
i. Find where are local maximal or minimal values related to tolerance : They can be found by zero first derivative. We say a point is extrema where its two-side points have different sign slope.

```
% since slopes of two sides near extrema point have different directions
df = p_x(1:end-1) - p_x(2:end);
cond = (df(1:end-1) .* df(2:end)) < 0;
% obtain the indices of extrema points
Ind_extrema = find(cond==true) + 1;
```

ii. The new alternating points include not only extrema but also start point, end point, passband frequency, and stopband frequency.

```
% obtain the indices of passband and stopband freq
[~,Ind_xp] = min(abs(xplotloc-x_p));
[~,Ind_xs] = min(abs(xplotloc-x_s));
% arrange indices
Ind = sort([1 Ind_extrema Ind_xp Ind_xs length(xplotloc)]);
```

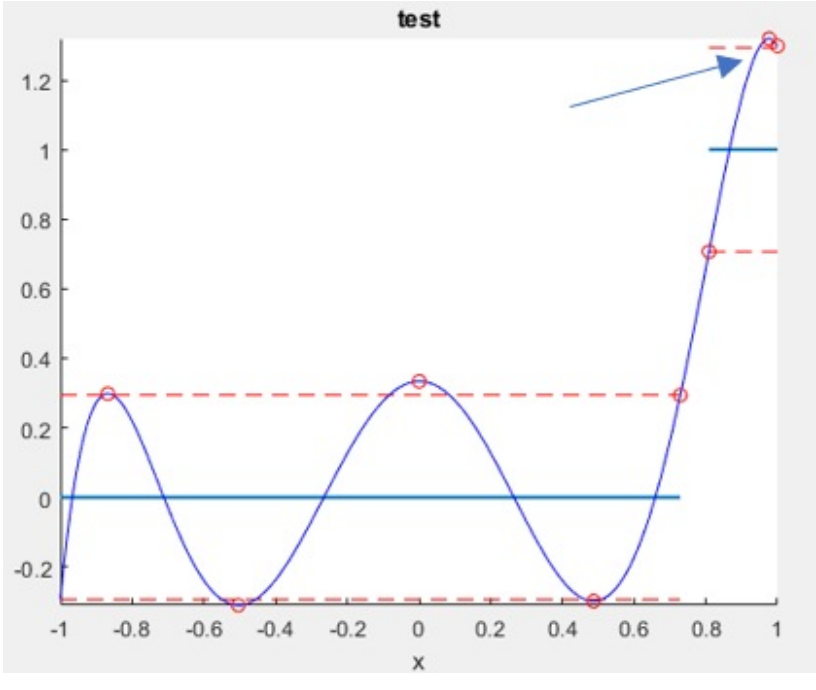
iii. Drop redundant extrema : Since we may simultaneously find many extrema points at the same side seen in figure below, these redundant extrema are required to drop out and hold only one. Extrema in passband and stopband should take into account respectively due to different base line, such as 1 for the former and 0 for the latter. We compare two adjacent points at a time and discard the smaller one if they are at the same side regarding to the base line. *Unique,* this function helps us to get rid of some issues, so does the *while* loop. We will discuss them in question 3.




```
% check where extrema points occur continuously
while true
    Drop_Ind = [];
    for ii = 2:(length(Ind)-1)
        % deal with alternating points in stopband
        if Ind(ii) <= Ind_xs
            % find the relatively larger one
            if abs(p_x(Ind(ii-1))) > abs(p_x(Ind(ii)))
                tmp = ii;
            else
                tmp = ii - 1;
            end
            % check whether or not adjacent points are invalid
            % if the two points are invalid, then drop the smaller o
            if p_x(Ind(ii-1))*p_x(Ind(ii)) > 0
                Drop_Ind = [Drop_Ind tmp];
            end
        % deal with alternating points in passband
        elseif Ind(ii) >= Ind_xp
            if abs(p_x(Ind(ii+1))-1) > abs(p_x(Ind(ii))-1)
                tmp = ii;
            else
                tmp = ii + 1;
            end
            if (p_x(Ind(ii+1))-1)*(p_x(Ind(ii))-1) > 0
                Drop_Ind = [Drop_Ind tmp];
            end
        % alternating points in transitionband is invalid
        else
            Drop_Ind = [Drop_Ind ii];
        end
    end
    % obtain indices
    Drop_Ind = unique(Drop_Ind);
    % drop them
    if ~isempty(Drop_Ind)
        Ind(Drop_Ind) = [];
    else
        break
    end
end
```

iv. Drop invalid points : if the terminal points, such as first point and last point, are located at the same side with adjacent point seen in the right figure, only the nearest extrema is retained.

```
% check where the start and end point follow rule of delta
passband_end = (p_x(Ind(end))-1)*(p_x(Ind(end-1))-1);
stopband_end = p_x(Ind(1))*p_x(Ind(2));
if passband_end > 0
    Ind = Ind(1:end-1);
end
if stopband_end > 0
    Ind = Ind(2:end);
end
```



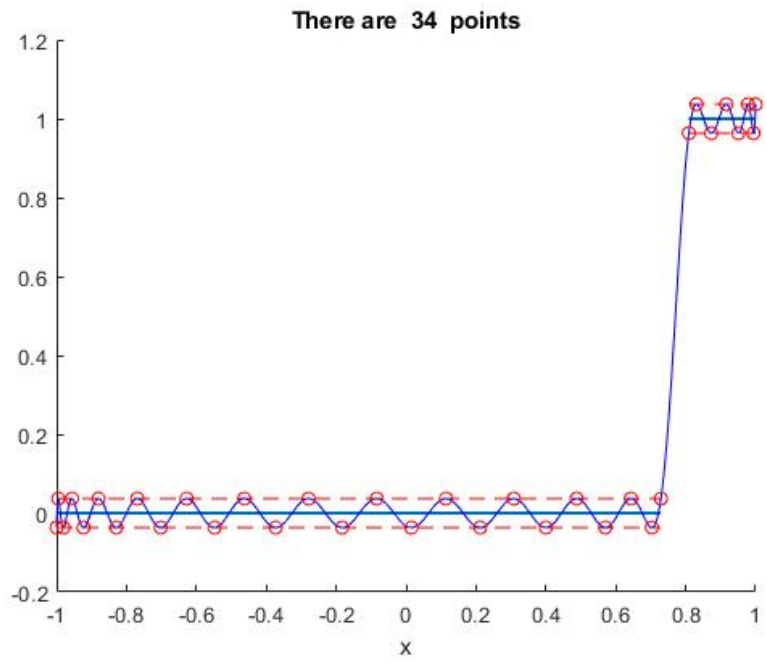
v. Drop redundant points : If number of new alternating point is still over L+2, we have to drop either start point or end point. Because other ones must be preserved to avoid violating the rule of tolerance. The discarded one is determined by which is much smaller related to the base line.

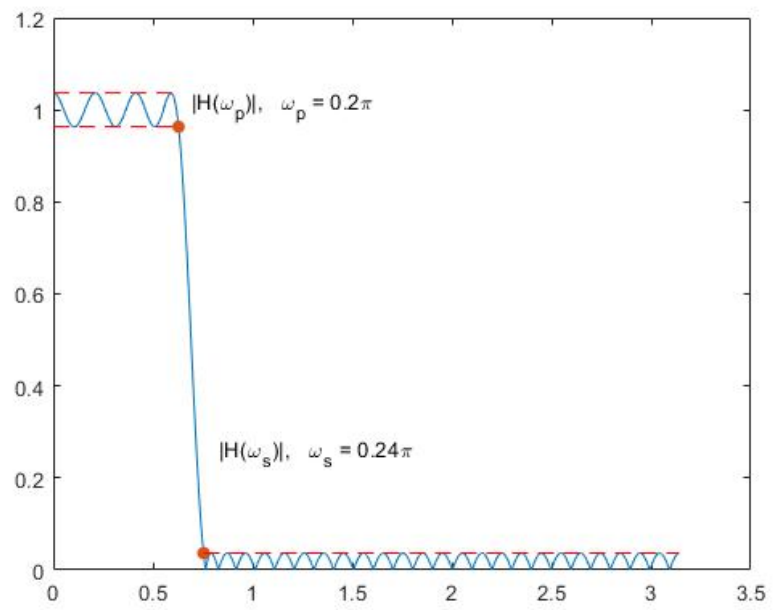
```
% if number of points is larger than L+2,then we drop either end or start
% one which is relatively smaller.
if length(Ind) > (L+2)
    if abs(p_x(end)-1)>abs(p_x(1))
        Ind = Ind(2:end);
    else

        Ind = Ind(1:end-1);
    end
end
```

2.

Thanking to TAs or professor, this part has been finished.

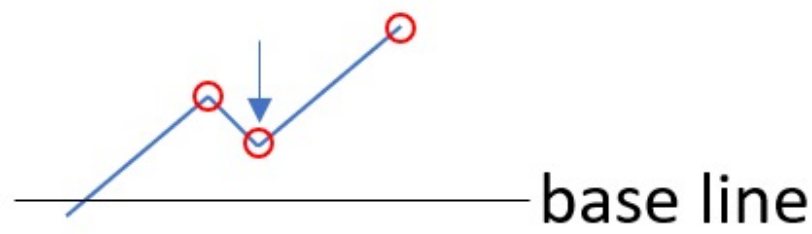
Original delta is 0.0104. After optimization, delta becomes 0.0367. It intuitively grows since we have to compromise with these extrema and need larger “room” to hold them.



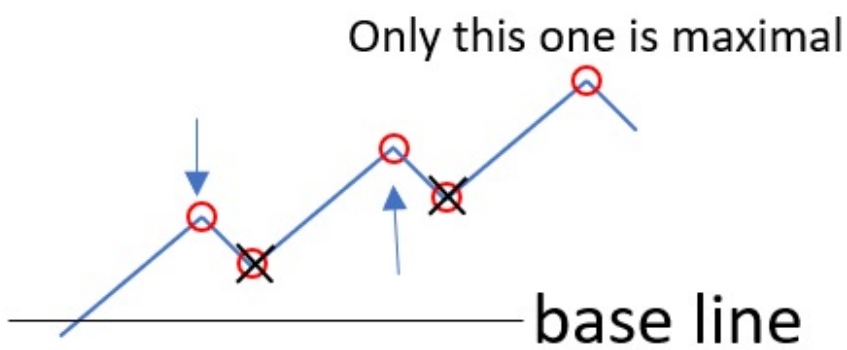


3.

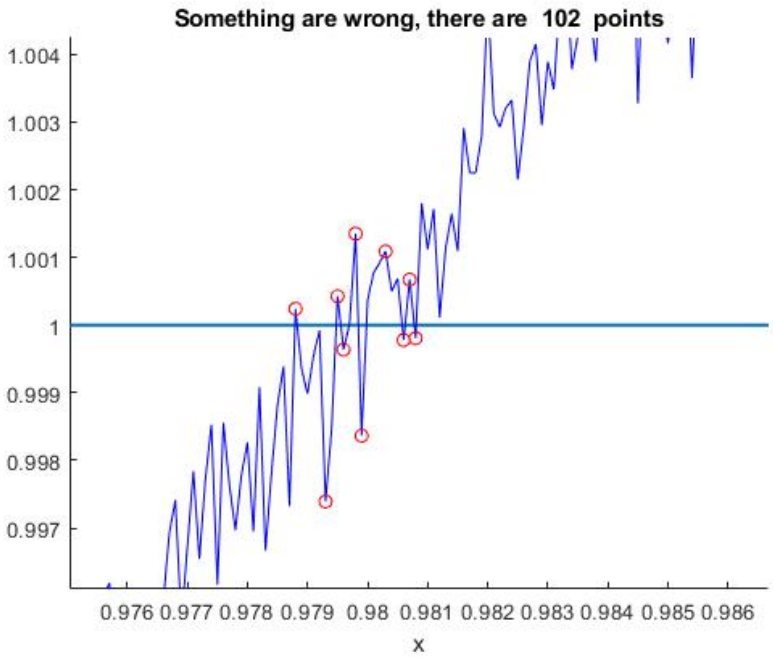
i. About *“Unique”* function : Since we check the extrema one by one, a point may be detected more than one time, just like the middle point shown in the following figure.

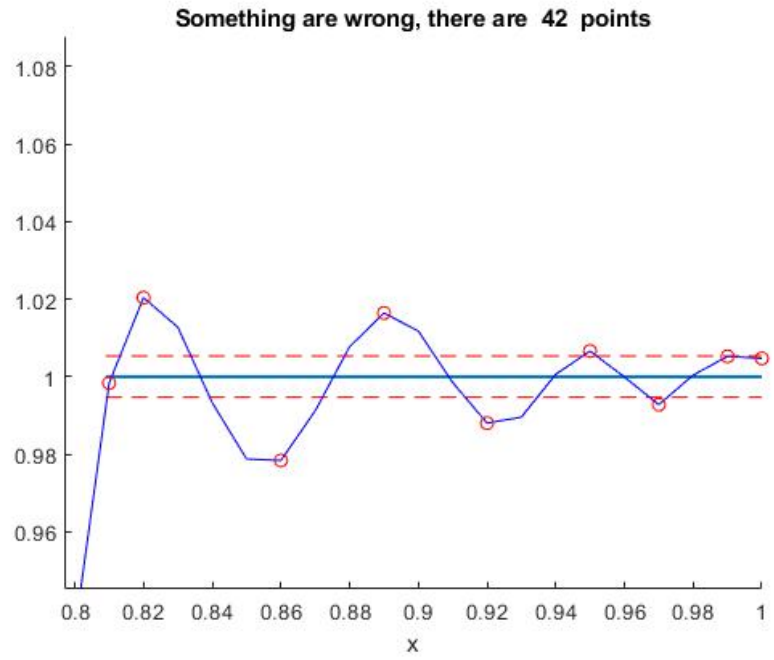


ii. About “while” loop : Sometimes these naughty points doesn’t occur only one time just like the following figure. The two block X terms will be eliminated and other three points will be preserved after our procedure without *while* loop. However, there must be only one extrema, so the other two ones pointed by arrow are still required to be discarded. That is why we need a loop to solve it.



iii. Spacing resolution dominates the quality of algorithm. Too small tick gives rise to serious jagged curve such that extrema happens rampantly seen in the left figure below. Too large tick leads to inaccurate coordinate seen in the right figure below. The reason for the former one might be due to round-off error, because we calculate *p_x* cumulatively. The two figures are set L as 40, but spacing resolution is 0.0001 and 0.01 respectively.





iv. Usually it doesn’t converge, such as L = 40, especially long length. In my opinion, this method, Parks-McClellan algorithm is not comprehensive. It has to suffice some condition otherwise can’t work well and struggle into dilemma.